

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name : M Kavya

Reg. No : 24011101061

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

GitHub Repository

7. Experimental Results

The Perceptron was trained on the Banknote Authentication dataset (80% train, 20% test split, learning rate $\eta = 0.01$, 30 epochs). The table summarizes the epoch-wise changes in errors, weights and bias.

Epoch	Errors	w_1	w_2	w_3	w_4	Bias
1	59	-0.0680	-0.0938	-0.0678	-0.0110	-0.0300
10	20	-0.1307	-0.1541	-0.1250	-0.0060	-0.0600
20	19	-0.1636	-0.1946	-0.1613	-0.0064	-0.0700
30	17	-0.1751	-0.2043	-0.1901	-0.0196	-0.0800

The misclassifications (error) count fell sharply from 59 in the first epoch to under 20 within the first 10 epochs, after which it didn't improve much and didn't converge to zero. This indicates that the classes are mostly but not perfectly linearly separable.

Test Set Performance

Metric	Value
Accuracy	0.9818 (98.18%)
Precision	0.9606
Recall	1.0000
F1-score	0.9799

Confusion Matrix :

$$\begin{pmatrix} 148 & 5 \\ 0 & 122 \end{pmatrix}$$

The classifier correctly identified every forged note in the test set (Recall = 1.0000, no false negatives) while it misclassified only 5 authentic notes as forged.

8. Generated Plots with Interpretation

8.1 Feature Histograms

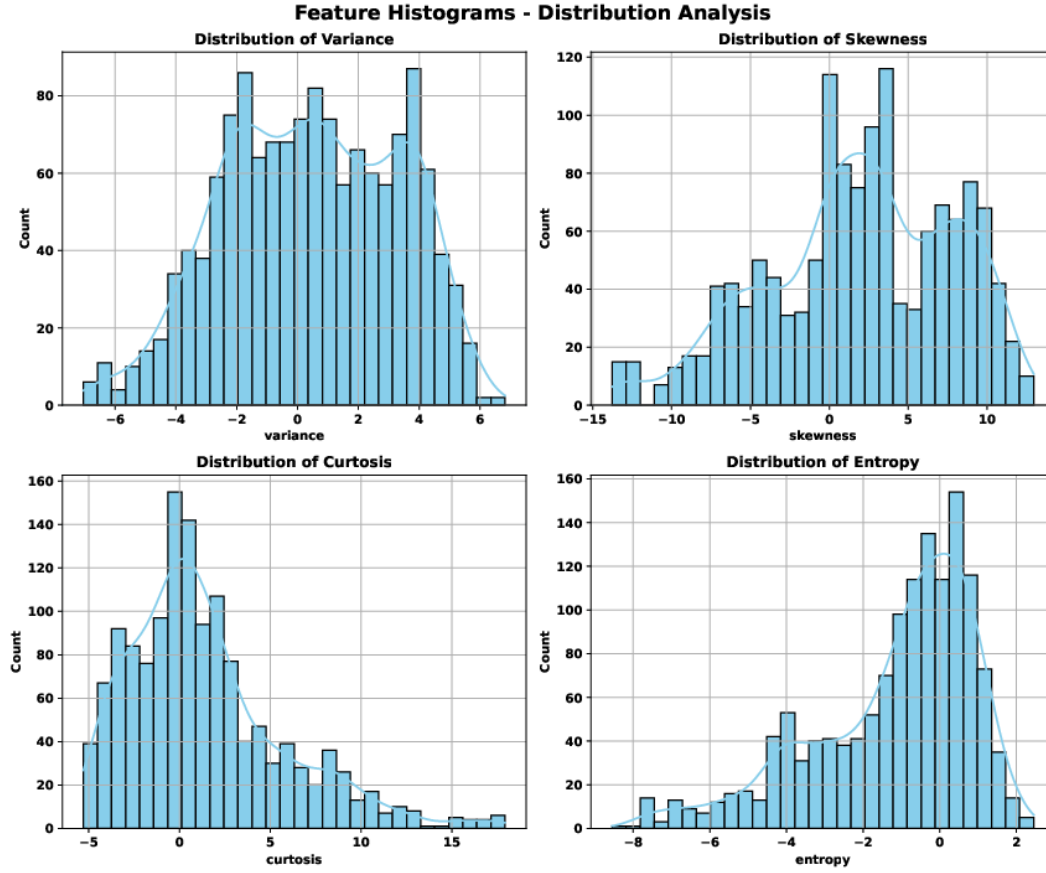


Figure 1: Distribution of Variance, Skewness, Kurtosis, and Entropy.

The histograms show the distribution of each input feature in the dataset. Variance has widest spread and is bimodal hence aids best in class separability. Entropy has most narrow spread so model learns least weight for this feature.

8.2 Correlation Heatmap

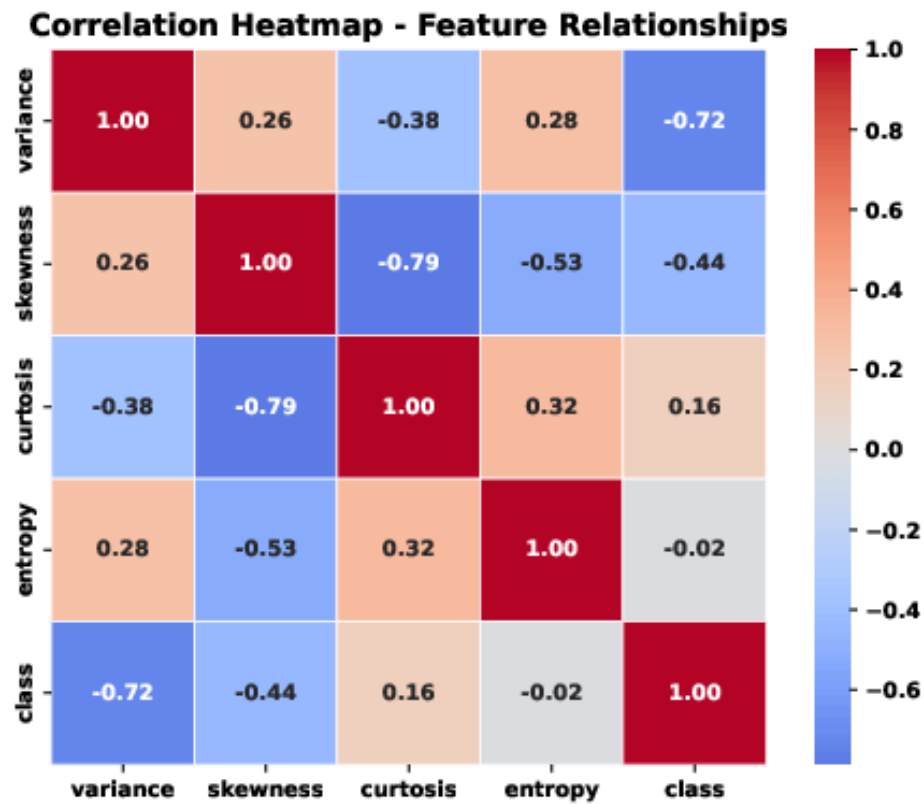


Figure 2: Pairwise correlation between the four input features.

The heatmap illustrates the correlation among the four numerical features. Kurtosis and skewness have a high negative correlation of -0.79 suggesting that they have an inverse relationship and the model doesn't need both features to contribute independently.

8.3 Scatter Plot

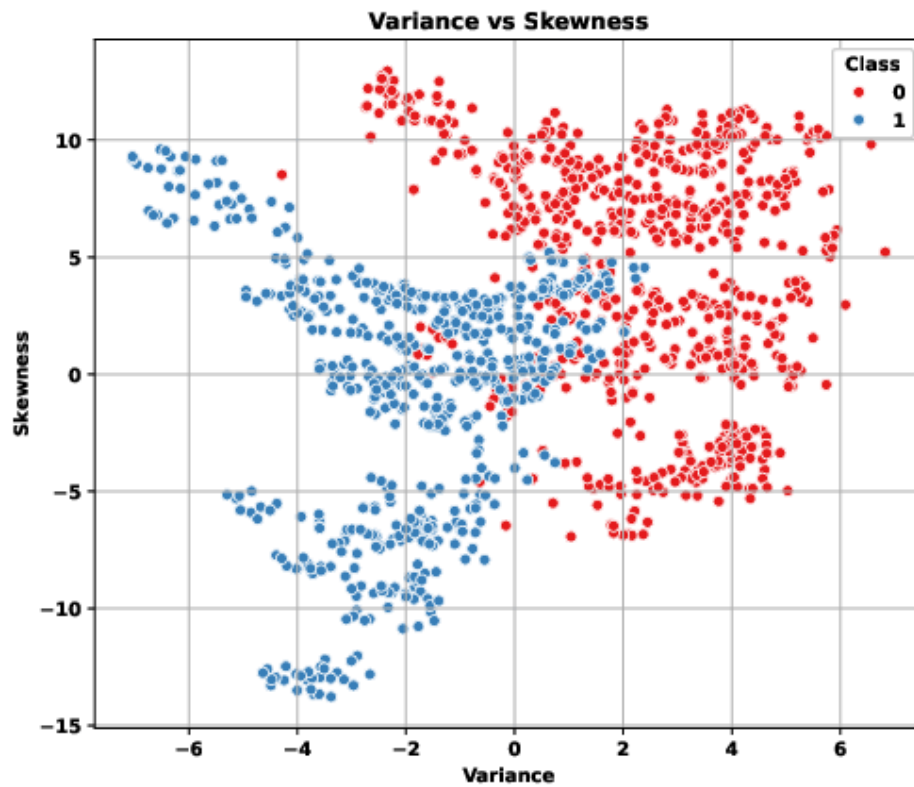


Figure 3: Scatter plot of Variance versus Skewness coloured by class.

The scatter plot shows that the classes are mostly separable but has a small diagonal overlap. This means that the classes aren't perfectly linearly separable.

8.4 Boxplots by Class

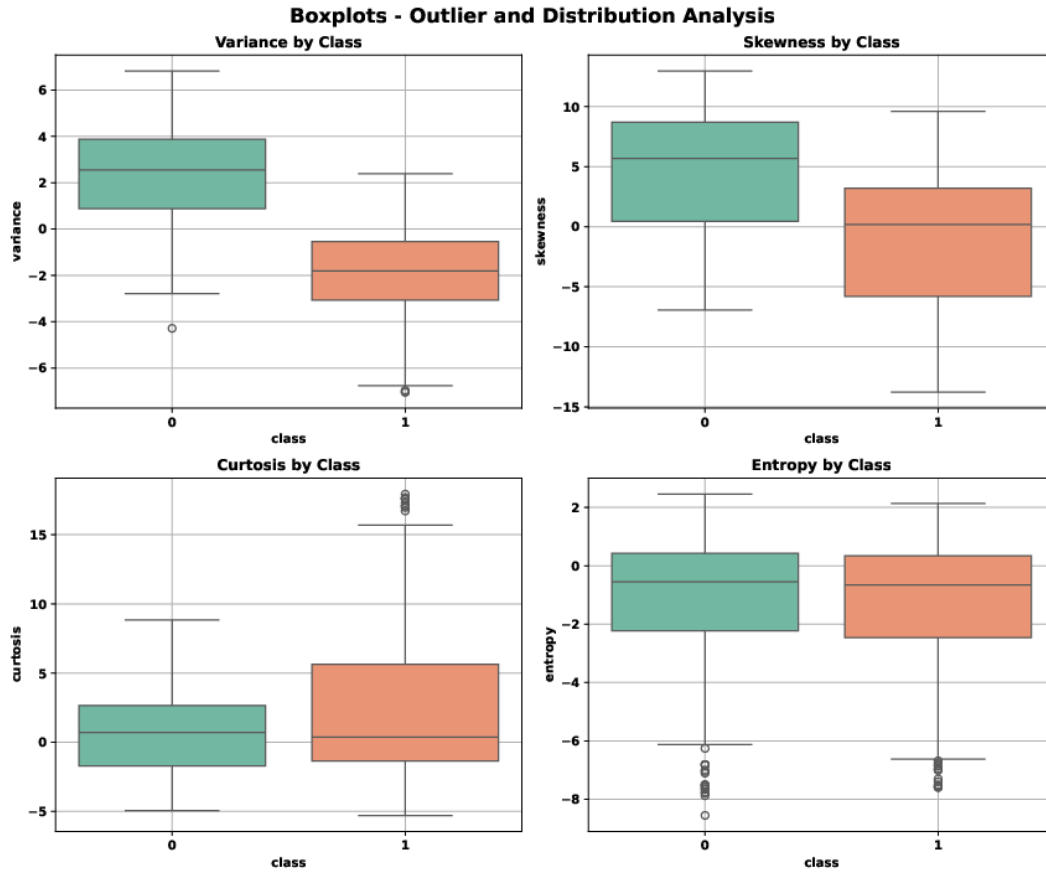


Figure 4: Feature-wise boxplots grouped by class.

The boxplots compare the distribution of each feature across the two classes. Variance shows least overlap between classes making it the strongest discriminator. Entropy shows highest overlap so its the weakest discriminator.

8.5 Training Error vs Epoch

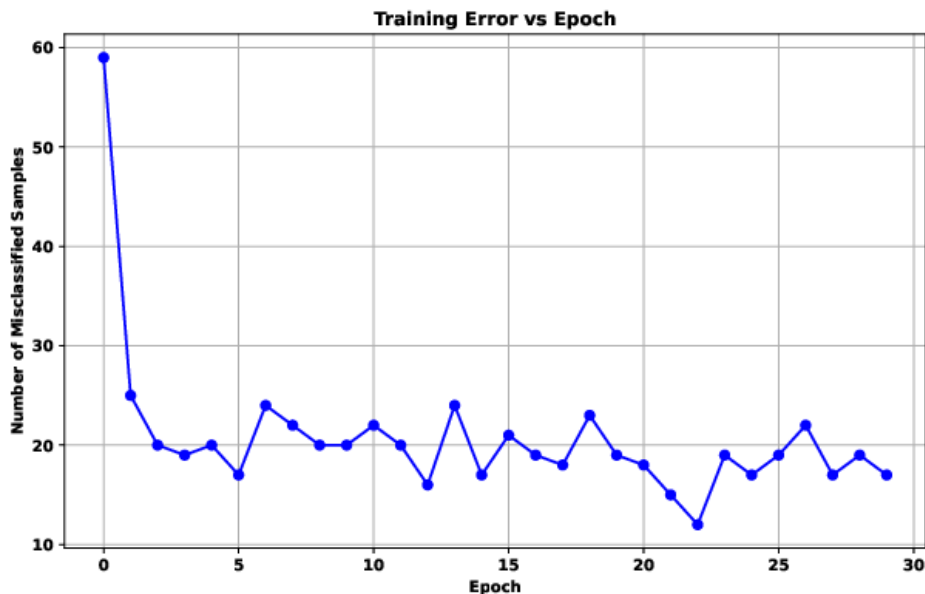


Figure 5: Number of misclassified samples per training epoch.

The number of misclassified samples decreases rapidly during the initial epochs. However it only reaches a minimum of 12 and never converges to 0 instead oscillates around the range of 12 and 25. This indicates the dataset is not fully linearly separable.

8.6 Weight Evolution

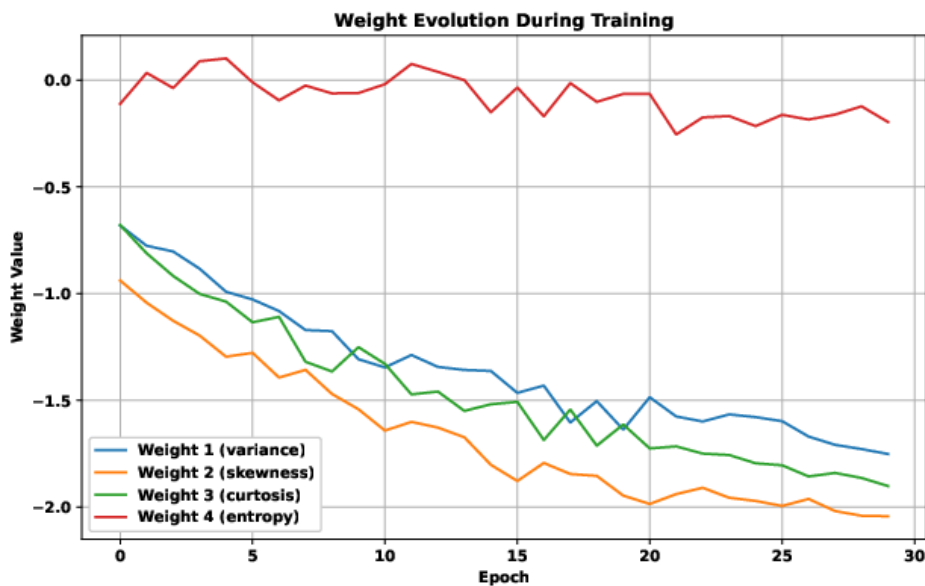


Figure 6: Evolution of the four feature weights across epochs.

Entropy remains low (near 0) throughout the epochs. The other feature weights drop rapidly in first few epochs but don't completely flat at the end as convergence is not completely reached.

8.7 Bias Evolution

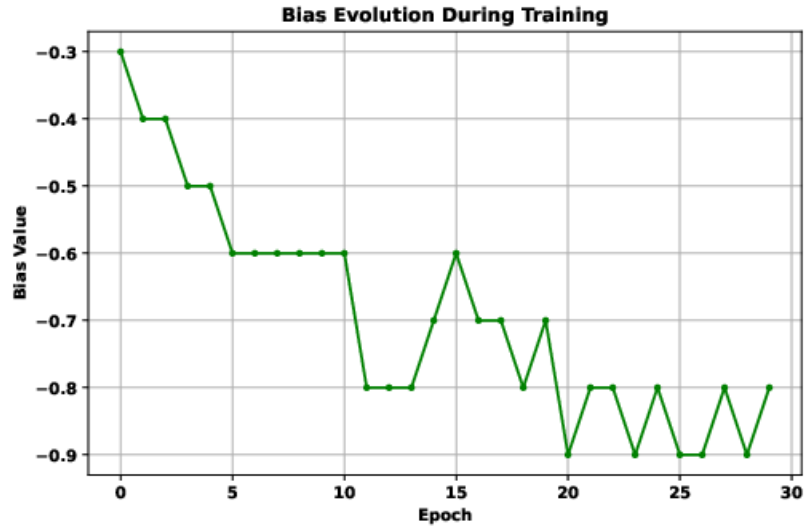


Figure 7: Evolution of the bias term across epochs.

The bias value drops significantly during the early epochs. It oscillates towards the end suggesting that full convergence has not been reached.

8.8 Confusion Matrix

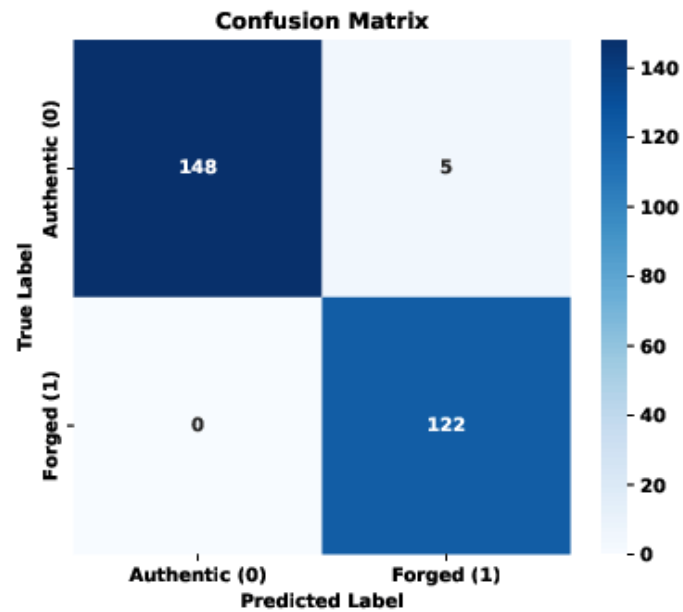


Figure 8: Confusion matrix on the held-out test set.

The confusion matrix indicates excellent classification performance with only a few misclassified samples. The model correctly identified all forged banknotes while misclassifying only 5 authentic notes.

8.9 Learning Rate Comparison



Figure 9: Training error versus epoch for different learning rates.

The comparison shows the model is insensitive to the learning rates chosen as the curves for all 3 are nearly identical. So the model is not very dependent on learning rates in the tested range.

8.10 Decision Boundary

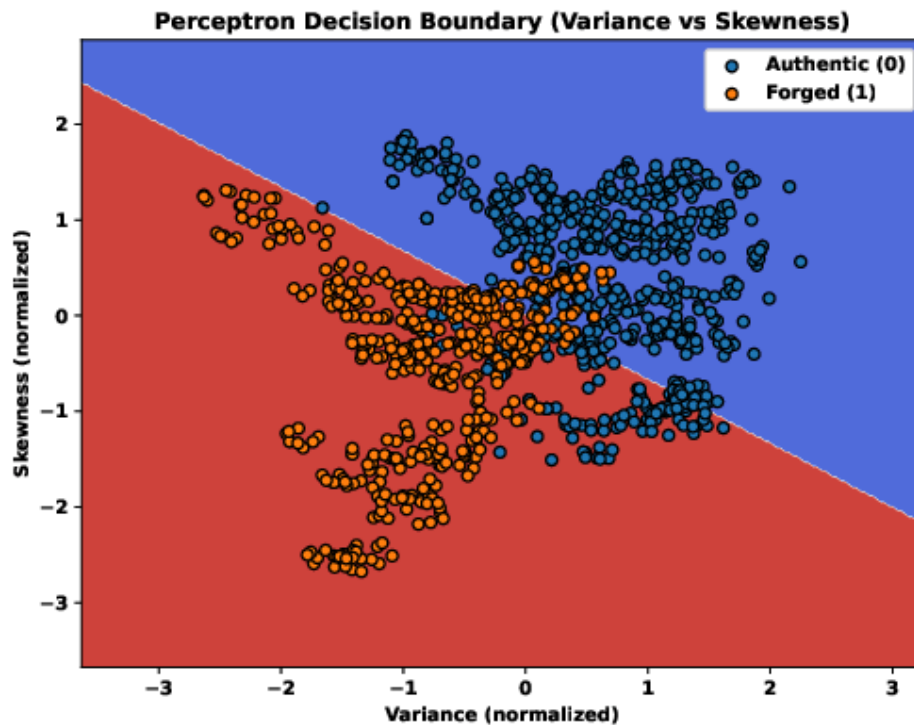


Figure 10: Decision boundary of the trained model.

The decision boundary shows how the line the model learnt to separate the two classes. The overlaps causing the misclassifications can be seen.

9. Performance Tables

Training Summary

Dataset Size	1372 samples (4 features)
Train/Test Split	80% / 20%
Learning Rate	0.01
Epochs	30
Final Weights (Best)	$[-0.1599, -0.1909, -0.1749, -0.0174]$
Final Bias (Best)	-0.0800
Accuracy	0.9818
Precision	0.9606
Recall	1.0000
F1-score	0.9799

Epoch-wise Learning

Epoch	Errors	w_1	w_2	w_3	w_4	Bias
1	59	-0.0680	-0.0938	-0.0678	-0.0110	-0.0300
15	17	-0.1362	-0.1802	-0.1518	-0.0150	-0.0700
30	17	-0.1751	-0.2043	-0.1901	-0.0196	-0.0800

10. Additional Tasks

1. **Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.**

The Step function outputs binary values (0 or 1) but is non-differentiable at $z = 0$. The Sigmoid function is smooth and differentiable, producing outputs in $(0, 1)$. So Step function cannot provide gradient signal for backpropagation while Sigmoid function can, enabling gradient descent.

2. **Compare the implementation with Scikit-learn's Perceptron.**

The from-scratch implementation provides full detailing of the learning rule and weight updates. Scikit-learn's Perceptron is faster and easier but the functionalities are hidden so we don't know how it works. But both achieved similar accuracy on this dataset.

3. **Repeat the experiment using learning rates 0.001, 0.01 and 0.1.**

All three learning rates performed the same indicating the model is very insensitive to changes in learning rates of this range.

4. **Explain why a Single Layer Perceptron cannot solve the XOR problem.**

A single-layer perceptron can only separate linearly separable data using a straight line (or hyperplane). The XOR problem is non-linearly separable, hence it cannot be solved by a single perceptron.

5. **Study the effect of feature normalization on model convergence.**

Feature normalization (StandardScaler) significantly improves convergence speed and stability by bringing all features to similar range, preventing dominance by high magnitude features.

6. **Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages.**

<https://github.com/Kavya-1709/Deep-Learning---Laboratory/tree/main/DL%20Lab%201%20-%20Single%20Layer%20Perceptron/OR%20%2C%20NOT%20and%20AND%20Gates> The three gates were implemented successfully since both OR and AND are linearly separable and a single perceptron is enough to learn them. NOT requires a simple decision rule hence that was also implemented easily.

11. Discussion

The from-scratch Perceptron achieved 98.18% test accuracy on the Banknote Authentication dataset indicating that the classes are almost, but not fully, linearly separable. The training-error curve

never reached 0 supporting the results of non-linear separability. The model only had a few misclassifications of authentic as forged and otherwise performed excellently. Overall the model was mostly successful.

12. Conclusion

A Single Layer Perceptron was successfully implemented from scratch and applied to binary banknote authentication, achieving good accuracy, precision, and perfect recall on the test set.

13. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.